LANCE

Where the project stands

Checkpoint 3: state report as of 3 November 2026

LANCE project

Written by specialist strands, independently reviewed and synthesised

Last revision: 2026-11-03 11:30 (Europe/Brussels, UTC+01:00)

Code at `546b911` (30 October 2026); later fixes (cost budget made observation-only, sandbox cleanup, dead-code removal) post-date it.

No new experiment is reported: the measurements cited are those already recorded.

Abstract

LANCE is a platform for auditing authorised connected-object networks with language-model agents, inside a laboratory. A scenario is deployed, an agent audits it, its evidence is archived, and what it found is scored against a ground truth. Developed in the context of the UMONS work of the NATO SPS G6259 programme.

Its benchmark is built to compare what one wants to compare, under the same conditions. This checkpoint documents one case, an automation against a model; the comparison that would separate them is not yet possible.

Nothing counts as a result until the project can point to a case where a rule written on one line gets the answer wrong and reasoning over several readings gets it right. Without such cases, nothing else the project does counts.

The scientific position is unchanged since the previous checkpoint — the platform is built, instrumented and documented, its chain of evidence sound — but the method advanced: a closure test now precedes each challenge, the inference is repaired, a reference H2 round was run, and a pilot challenge was built and rejected.

Contents

1 Where the project stands **2**

2 What is built, and what is measured **4**

3 What remains **6**

4 Conclusion **7**

1 Where the project stands

1.1 The project in one paragraph

LANCE lets language-model agents audit IoT networks. From a description it builds the network to audit, hands the scenario to an agent, archives everything the agent does, and scores the result against a ground truth fixed in advance. The point is not that a model should win, but that an audit carried out by an agent can be run, re-run and verified.

The benchmark compares what one wants to compare under the same conditions; this checkpoint documents one case, an automation against a model. Comparing the two is not possible yet: no challenge has yet separated them enough to tell them apart, and the policy that holds everything constant but the interpretation does not exist (Figure 2). The platform is built, the comparison is half instrumented, and the material to measure it has not been found.

1.2 What an audit does, and where the model decides

An audit runs in six phases. Each phase produces one document that the next one reads, and nothing else crosses between them. Figure 1 is the map to keep in mind for the rest of the report: it shows the six phases, what each produces, and the point that matters: *who* decides at each step.

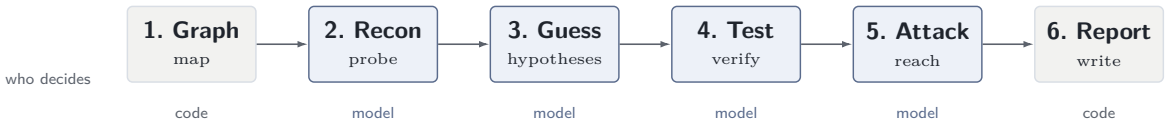


Figure 1 – An audit in six phases, and where the model's freedom actually sits. The model decides at phases 2, 3, 4 and 5: which probes to run, which weaknesses to hypothesise, how to test each one, and how far to push an intrusion. What the code decides is what becomes verifiable: phase 1 projects its evidence file from the graph tools and refuses a submission that misses a device. When the model's deliverable at phase 5 is missing or invalid, the code writes the assessment from the tool ledger, and a completed phase never certifies that the objectives were reached. Phase 6 assembles the report from section sheets, with an agent that declares no tool and runs a single turn. The score is produced elsewhere, by an offline evaluator that recounts the log against an oracle fixed before the run.

The log is the source, not the model's conclusion. Every tool call is archived with a reference, a refusal or an exception included. A finding that cites a piece of evidence which does not appear in the log is rejected. This is what makes a result attributable: the report states what happened, not what the policy believed happened.

The oracle precedes the run. The expected answer is fixed before the policy runs, in a file the policy never receives. A finding that contradicts the oracle invalidates the run instead of correcting it. This precedence is what prevents the answer from being adjusted after the fact.

The score is recomputed, never taken on trust. A finding counts as found only if the evidence it cites appears in the journal. The SSH, MQTT, API and V0 evaluators receive only the briefing, the oracle, the journal and the final decision, and they return an unknown outcome when a cited observation is missing. The benchmark score of a pipeline run does not perform that check; the confirmed metrics it would need are not emitted by the harnesses today.

Where this lives. Phase registry `src/agent/phases/registry.py`; governance boundary `src/agent/core/executor.py`; evaluators under `src/benchmark/` and `benchmarks/experiments/`.

1.3 What runs in the laboratory

Inside the laboratory, a deployed scenario is handed to a policy and its result scored against the ground truth fixed in advance. Three policies are in question here: versioned procedures, the scanner followed by its rules, and a model given the scanner's observations alone. All three receive the same scenario and the same observations; what differs is what they are allowed to do with it. The third does not exist yet. Only the full pipeline earns a benchmark score; the rules policy still carries a strict-v3 score and the scripted one carries none, so a comparison across policies needs an evaluator all of them go through.

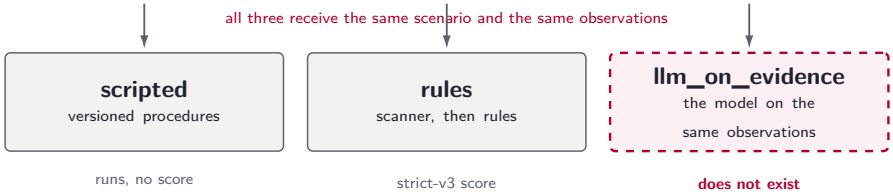


Figure 2 – The three policies and what is missing. The third would give the model the observations the rules already receive and nothing else: no rule findings, no ground truth, no expected count of flaws. It does not exist, and without it a comparison has two policies and measures verification rather than interpretation.

What is missing, and why it matters. An exploration where the model roams a network freely changes too much at once: the targets chosen, the order of the probes and the interpretation all differ, so a better score cannot be attributed to reasoning. Giving the model the scanner's observations isolates one variable: *who interprets*. That is the question the programme asked, and the third policy is the only one that answers it.

1.4 What the benchmark measures

A comparison needs a unit of observation, or ten similar scenarios count as ten proofs. Three nested terms keep the count honest, and only the outermost one counts.

— **A scenario** is one deployment that gets audited: the devices, the services, the injected flaw and the links between them. It is the matter, and it counts for nothing on its own (Figure 3).

— **A series** groups scenarios that differ by a single axis — here, only the kind of vulnerable machine changes. It is a grouping, not a proof, and it too counts for nothing on its own.

— **A challenge** is what is submitted to the closure test: one or more series. It counts as *one* observation if it resists, whatever the number of series, scenarios and runs it holds. Two challenges share nothing: not a simulator, not a vocabulary, not a fault model.

The number of runs does not enter the count: the same scenario audited four times by the same model remains one data point, not four.

Earlier reports said *family*, and the code still does: `family_id` names a challenge and `trial_id` names one of its runs. The prose is retired because the word named the grouping and never what the grouping had to prove; the code identifiers stay, naming data, not the vocabulary.

What the closure test is. Before building a challenge, one asks whether a rule that solves the case can be written in a single line. If it can, the challenge measures the omission of a declaration rather than a limit of interpretation, and it is not built. The test is cheap; running it before writing eight fixtures is the change of method this month produced.

The choices that follow from this state are stated in `objectifs/ETAT-COURANT.md`; none of them is taken yet.

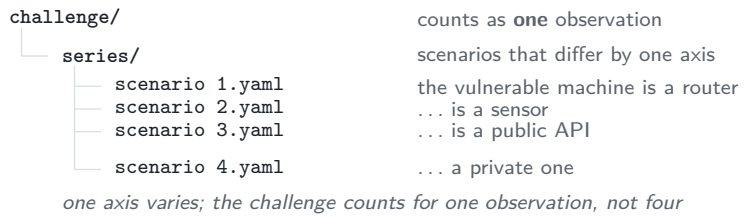


Figure 3 – What a challenge contains, nested. A **scenario** is one deployment, and counts for nothing on its own. A **series** groups scenarios that differ by a single axis — here, only the kind of vulnerable machine. A **challenge** is what is submitted to the closure test: it counts as one observation, whatever the number of series, scenarios and runs it holds.

1.5 The state in one table

Table 1 – The state of the project on 3 November 2026, and where the proof is.

Question	Answer	**Proof**
Is an audit traceable?	Yes. Every tool call is archived with a reference, bounded, and a piece of evidence that cannot be archived stops the run instead of leaving a silent gap. The guards are application-level.	table 2
Does the harness run outside the laboratory?	Yes, without broker and without a model, under a guard whose blind spots are declared. It instruments a contrast; it measures nothing.	table 2
Are the protocols' contracts sound?	Yes. Briefing, private oracle and offline evaluator are separated, and the oracle is fixed before the run.	table 2
Can results be delivered reproducibly?	Yes. Push, validate, deploy to one of three instances; no execution on a workstation. Single points of failure declared.	table 2
Does the model beat the rules?	**Unknown, and not yet measurable.**	Section 2
How much of the corpus needs interpretation?	84.4 % is rule-decidable (interpretive reading); 282 of 288 by signature (binary reading); two counts from two artefacts, never cited one for the other.	Section 2
Is the measurement statistically valid?	**No:** the tool is repaired but needs five independent challenges, and none exist.	Section 2

2 What is built, and what is measured

In brief. Nine components exist, each with a stated guarantee and a stated limit. None of them proves that a model does better than a script, which is the object of the programme. The detail is in `docs/` for the behaviour and in the archived reports for the implementation; what follows is what a committee needs in order to know whether the instruments can be trusted.

2.1 What is measured, and what the measures say

The fidelity gate passes, partially. The V0 control, a task solvable without inference, succeeds on both engines three times out of three, with a saturating control: it is readable for a repeated defect, not for a rare one. The threshold frozen at 0.9 is computed separately by the gate and conditions nothing.

One challenge was measured, and its gain was absorbed. H2, on 22 October 2026: eight fixtures, 25 messages, two readable measurement rounds. The first ran the two engines of the time; the reference round that replaced it rotates three engines over three repetitions across the eight

Table 2 – What exists, what it guarantees, and what it does not.

Component	Guarantee	Limit
Execution boundary	every tool call is archived with a reference, bounded by budgets, refused outside the target network	application-level guards, not a system sandbox: the destination check is a consultative IPv4 parser, and a tool carrying its destination in an unscanned field (`dig`, `redis`, `ysoserial`) is not refused
Laboratory	a scenario deployed, configured and reset, then destroyed: a run starts from the same state	one host, one container, one runner
Delivery	push, validate, deploy to one of three instances; no execution on a workstation	no automatic rollback after a failed deployment
Audit pipeline	six phases, one document per phase, the model decides where interpretation is needed (Figure 1)	two scenarios on one laboratory are not mutually excluded
Evidence contracts	briefing, private oracle and offline evaluator separated; the oracle is fixed before the run	four evaluator contracts only
Harnesses	one contract, one deterministic engine, one model engine, one evaluator per contract	the H2 engines are offline; other controllers need the laboratory
Sandbox	the deterministic engines run without broker, model or laboratory, under a guard whose blind spots are declared	it instruments a contrast, it measures nothing
Corpus	288 ground-truth entries over the catalogue	built around what signature rules find (table 3)
Inference tool	one-sided p-value against the pre-registered threshold, no figure below three challenges	it has nothing to compute on

fixtures, which gives 72 runs. On the raw variants the model scores 0.5, the declared parser 0.0, and a third engine reading by convention covers 8 of 8: it absorbs the gain without declaring anything. This model engine is not the `llm_on_evidence` policy of Figure 2: it is H2's offline format engine, not the model given the scanner's observations alone, so the comparable policy remains missing.

One challenge was built and rejected in an hour. The vendor-register probe of 30 October resisted six recorded reviews and fell to a rule that reads the effect of a write on the neighbouring registers. The filter that would have caught it costs minutes: if the rule settling the case can be written in one line, a script will have it.

The inference is repaired and idle. The tool now computes a one-sided p-value against the threshold, refuses to emit a figure below three challenges and requires five to decide. No set of five independent challenges exists, so the arithmetic has nothing to work on.

The exploratory pilot published three pairs, not twelve. The reconciliation of 16 October plans twelve pairs. Eight attempts across seven pairs were executed under a freeze: three were published as comparable and five were incomparable. The campaign is not admissible.

2.2 The corpus, and why it cannot answer the question

Table 3 – Two counts, from two artefacts, never to be cited one for the other.

Reading	Decidable by a rule	Not decidable	Total
Mechanical (binary `type` of each entry)	282 by signature	6 by logic (2.1 %)	288
Interpretive (category judged as a whole)	243 (84.4 %)	35 mixed, 4 chaining, 6 logic	288

Both are recomputed from `benchmarks/labels/derived_index.json` for the mechanical reading and from the coverage audit of 12 October 2026 for the interpretive one, which totals categories rather than entries. A duel on this corpus would rank rules against rules, which is exactly what H2 observed.

3 What remains

3.1 What the project can assert, and what it cannot

An audit produces verifiable traces: every tool call is archived and referenced, and the run stops when its evidence log cannot be written. The guards, however, are application-level. A harness can run outside the laboratory, without broker or model, under a guard that declares its blind spots; it instruments a contrast and measures nothing. A third engine, reading by convention, takes a gain that was attributed to the model, but that holds for eight fixtures and one property. And the corpus is mostly detectable by signature, so a null difference is expected by construction, though that concerns this corpus and not the situations the programme targets.

Must not be written as established: the superiority of the model; a significant gap, since no set of five independent challenges exists to compute one; generality, H2 covering one schema, one question and one model; a cost gain; the validity of the V0 threshold, which the gate does not apply; and the integrity of the corpus, the official healthy control being absent from the catalogue.

The shortfall is not only one of count. The challenges that exist are fabricated: each catalogue scenario carries its own identifier as a challenge, so at best three are defensible against the five the inference requires. Until challenges are built for the question rather than around what the rules already find, the corpus cannot answer it.

3.2 What would settle each open point

Each open point is paired in Table 4 with the observable test that would show it settled, which is the one thing a table of states cannot carry. Three of the four depend on the first: a comparable engine, a written reference automation and a corpus built for the question are worth building only once a situation exists that a one-line rule does not settle. Before that, each of them measures the absence of that situation. **The first line is the only work that can start today.**

Table 4 – Each open point, what would settle it, and how one would know.

Open point	What would settle it	How one would know
No situation a rule cannot decide	a case where no one-line rule settles the answer	the closure script runs and refuses to admit it
No comparable engine	the model interpreting the very observations the rules receive	the same scenario scored on both, by the same evaluator
The competent reference is a doctrine	a written reference automation standing in for it	it passes a challenge that the model also passes
Corpus built for rules	scenarios where a well-written rule can only abstain	the interpretive share rises, recomputed entry by entry

This is where the report stops. Everything up to a verified measurement is built; the thing to be measured is not — and of the four open points, it is the only one whose proof needs no new agreement on method.

4 Conclusion

The system is built, instrumented and documented. Its chain of evidence is sound: a separate oracle, an evaluator that recounts, a run that stops when its evidence cannot be archived, a safeguard that declares its blind spots. The verification discipline works: every claim here was checked against the code, not against what it says about itself.

The scientific question remains open. The programme asked when a model does better than a script with identical tools; the available data do not answer it, and the current arrangement would not allow an answer even with more machine time. The next result the project can legitimately produce is *a challenge that survives its closure test*, not a better score.